NIR2FPGA: Compiling Spiking Neural Networks into RTL

Michail Rontionov

University of Southampton

30th April 2026

Hypothesis 1

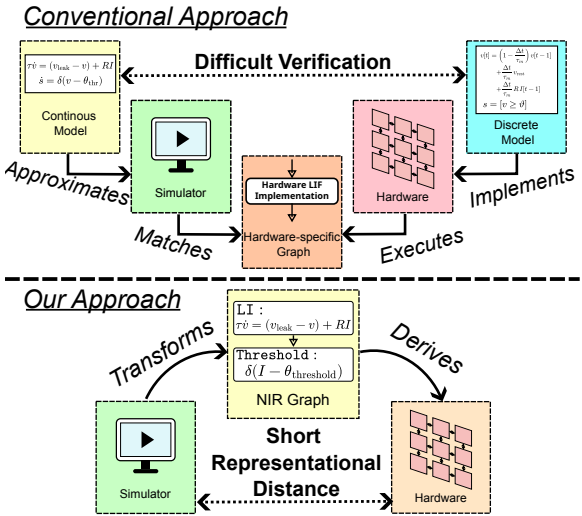
H_1

In the context of neuromorphic FPGA/ASIC co-design, it is **enough to implement the primitives and connections** between them to **describe the majority of** high-level **SNNs** whilst meeting a given accuracy metric.

Hypothesis 1

H_1

In the context of neuromorphic FPGA/ASIC co-design, it is **enough to implement the primitives and connections** between them to **describe the majority of** high-level **SNNs** whilst meeting a given accuracy metric.

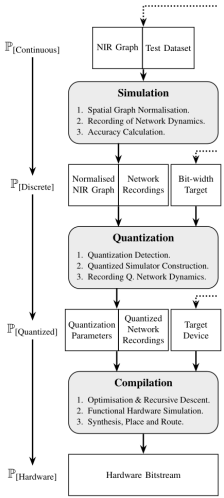


NIR2FPGA: Generating Dataflow Accelerators from the Neuromorphic Intermediate Representation

Michail Rontionov[1†], Francisco Ayala Le Brun[2], Nassim Beladel[3],
David Thomas[1], Charlotte Frenkel[4], Jens Egholm Pedersen[5‡]
[1]University of Southampton, Southampton, UK [2]Independent Researcher, Netherlands
[3]ETH Zürich, Zürich, Switzerland [4]Delft University of Technology, Netherlands
[5]Technical University of Denmark
Emails: † m.rontionov@soton.ac.uk, ‡ jegpe@dtu.dk

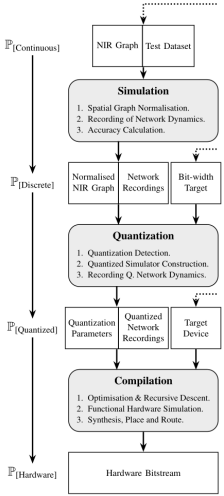
What does it do?

- Example scenario: "I have an SNN with the *CuBa-LIF* neuron model, implemented in a *NIR-supported simulator*. I would like to evaluate its edge performance in hardware".



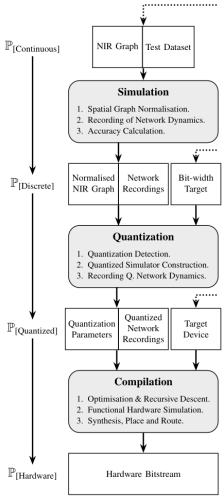
What does it do?

- Example scenario: "I have an SNN with the *CuBa-LIF* neuron model, implemented in a *NIR-supported simulator*. I would like to evaluate its edge performance in hardware".
- Questions arising from using other FPGA works: What source simulators do they assume? Which differential model did they use? Which discretization method did they use? How can I optimise their quantization for my network?



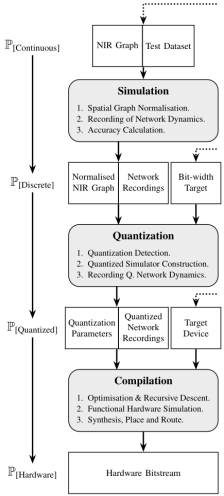
What does it do?

- Example scenario: "I have an SNN with the *CuBa-LIF* neuron model, implemented in a *NIR-supported simulator*. I would like to evaluate its edge performance in hardware".
- Questions arising from using other FPGA works: What source simulators do they assume? Which differential model did they use? Which discretization method did they use? How can I optimise their quantization for my network?
- Inputs:



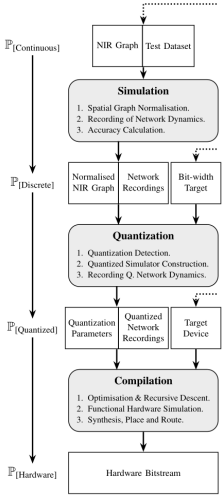
What does it do?

- Example scenario: "I have an SNN with the *CuBa-LIF* neuron model, implemented in a *NIR-supported simulator*. I would like to evaluate its edge performance in hardware".
- Questions arising from using other FPGA works: What source simulators do they assume? Which differential model did they use? Which discretization method did they use? How can I optimise their quantization for my network?
- Inputs:
  - NIR Graph (exported from simulator).



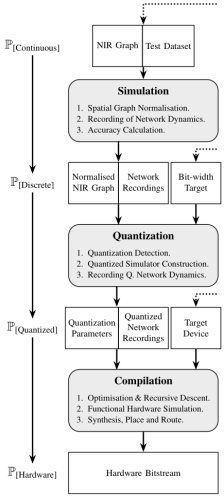
What does it do?

- Example scenario: "I have an SNN with the *CuBa-LIF* neuron model, implemented in a *NIR-supported simulator*. I would like to evaluate its edge performance in hardware".
- Questions arising from using other FPGA works: What source simulators do they assume? Which differential model did they use? Which discretization method did they use? How can I optimise their quantization for my network?
- Inputs:
  - NIR Graph (exported from simulator).
  - Dataset.



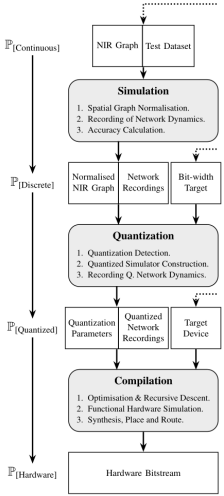
What does it do?

- Example scenario: "I have an SNN with the *CuBa-LIF* neuron model, implemented in a *NIR-supported simulator*. I would like to evaluate its edge performance in hardware".
- Questions arising from using other FPGA works: What source simulators do they assume? Which differential model did they use? Which discretization method did they use? How can I optimise their quantization for my network?
- Inputs:
  - NIR Graph (exported from simulator).
  - Dataset.
  - Discretization choices (discretization method (e.g. Euler), quantization options, bit width targets)



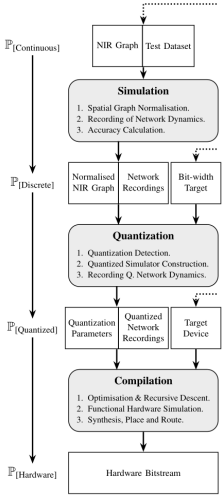
What does it do?

- Example scenario: "I have an SNN with the *CuBa-LIF* neuron model, implemented in a *NIR-supported simulator*. I would like to evaluate its edge performance in hardware".
- Questions arising from using other FPGA works: What source simulators do they assume? Which differential model did they use? Which discretization method did they use? How can I optimise their quantization for my network?
- Inputs:
  - NIR Graph (exported from simulator).
  - Dataset.
  - Discretization choices (discretization method (e.g. Euler), quantization options, bit width targets)
- Outputs:



What does it do?

- Example scenario: "I have an SNN with the *CuBa-LIF* neuron model, implemented in a *NIR-supported simulator*. I would like to evaluate its edge performance in hardware".
- Questions arising from using other FPGA works: What source simulators do they assume? Which differential model did they use? Which discretization method did they use? How can I optimise their quantization for my network?
- Inputs:
  - NIR Graph (exported from simulator).
  - Dataset.
  - Discretization choices (discretization method (e.g. Euler), quantization options, bit width targets)
- Outputs:
  - Hardware Bitstream according to inputs.



Theory

We make the definitions

- A **primitive** $p \in \mathbb{P}$ is defined by its inputs, parameters, bounding differential equations and outputs.

Theory

We make the definitions

- A **primitive** $p \in \mathbb{P}$ is defined by its inputs, parameters, bounding differential equations and outputs.
- A **computational model** $\mathcal{M}$ specifies what regime primitive p is evaluated in, for example: *ODE*, *Discrete* equation, *Quantized* equation with discrete model, or in a digital *Circuit*.

Theory

We make the definitions

- A **primitive** $p \in \mathbb{P}$ is defined by its inputs, parameters, bounding differential equations and outputs.
- A **computational model** $\mathcal{M}$ specifies what regime primitive p is evaluated in, for example: *ODE*, *Discrete* equation, *Quantized* equation with discrete model, or in a digital *Circuit*.
- A **primitive mapping** $\mathbb{P}_{[\mathcal{C}]}$ maps a primitive set $\mathbb{P}$ into a computational model $\mathcal{M}$.

Theory

We make the definitions

- A **primitive** $p \in \mathbb{P}$ is defined by its inputs, parameters, bounding differential equations and outputs.
- A **computational model** $\mathcal{M}$ specifies what regime primitive p is evaluated in, for example: *ODE*, *Discrete* equation, *Quantized* equation with discrete model, or in a digital *Circuit*.
- A **primitive mapping** $\mathbb{P}_{[\mathcal{C}]}$ maps a primitive set $\mathbb{P}$ into a computational model $\mathcal{M}$.

The paper aims to implement the transformation

$$\mathbb{P}_{[ODE]} \rightarrow \mathbb{P}_{[Circuit]}$$

Theory

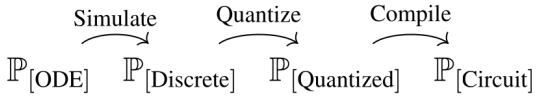
We make the definitions

- A **primitive** $p \in \mathbb{P}$ is defined by its inputs, parameters, bounding differential equations and outputs.
- A **computational model** $\mathcal{M}$ specifies what regime primitive p is evaluated in, for example: *ODE*, *Discrete* equation, *Quantized* equation with discrete model, or in a digital *Circuit*.
- A **primitive mapping** $\mathbb{P}_{[\mathcal{C}]}$ maps a primitive set $\mathbb{P}$ into a computational model $\mathcal{M}$.

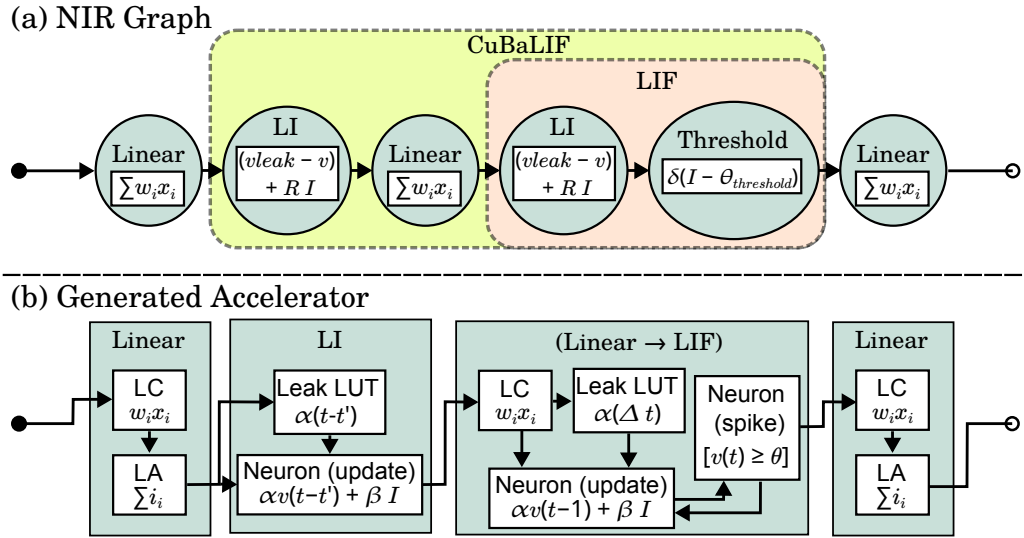
The paper aims to implement the transformation

$$\mathbb{P}_{[ODE]} \rightarrow \mathbb{P}_{[Circuit]}$$

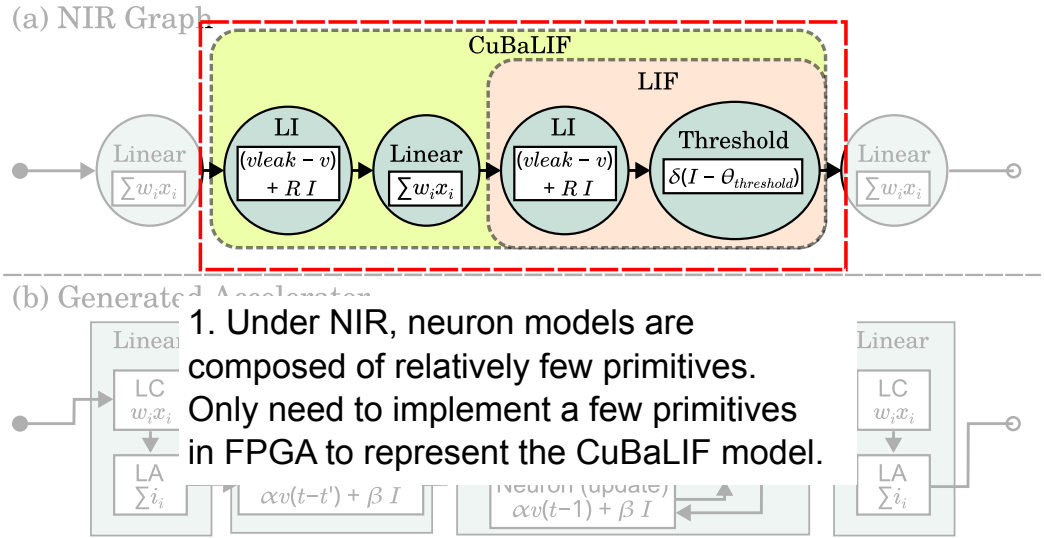
We model this as three separate unidirectional transformations



Example Generation

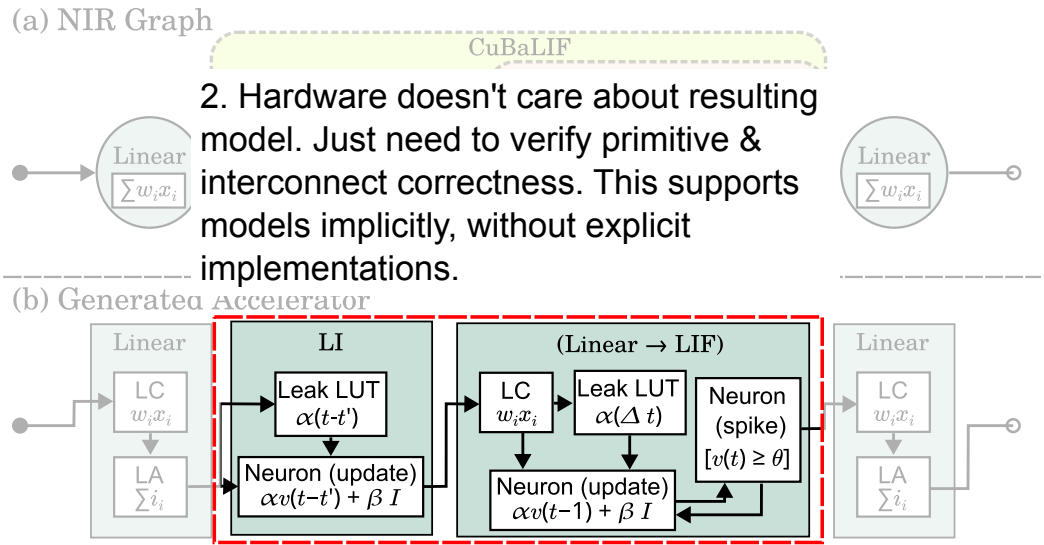


Example Generation



1. Under NIR, neuron models are composed of relatively few primitives.
Only need to implement a few primitives in FPGA to represent the CuBaLIF model.

Example Generation

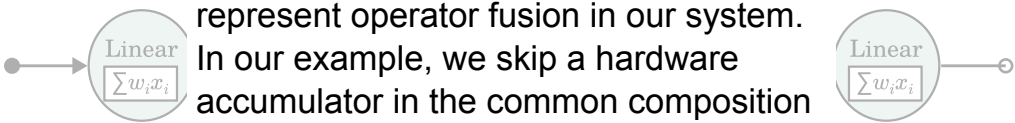


Example Generation

(a) NIR Graph

CuBaLIF

3. Assuming NIR, we can effectively represent operator fusion in our system. In our example, we skip a hardware accumulator in the common composition Linear -> LIF.



(b) Generated Accelerator